

PRAKTIKUM SISTEM OPERASI

MODUL 7
(Semaphore)



Oleh:

(Ammar Dzaki Nandana)

(2311104071)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

MODUL 7

1. Implementasi Sinkronisasi Menggunakan Semaphore

Kode Program

```
#include <xinu.h>

sid32 s1, s2, s3;

process p1(void) {
    while (1) {
        wait(s1);
        printf("Kwak\n");
        signal(s2);
    }
}

process p2(void) {
    while (1) {
        wait(s2);
        printf("Kwik\n");
        signal(s3);
    }
}

process p3(void) {
    while (1) {
        wait(s3);
        printf("Kwek\n");
        signal(s1);
    }
}

process main(void) {

    s1 = semcreate(1);
    s2 = semcreate(0);
    s3 = semcreate(0);

    resume(create(p1, 1024, 20, "P1", 0));
    resume(create(p2, 1024, 20, "P2", 0));
    resume(create(p3, 1024, 20, "P3", 0));

    return OK;
}
```

Penjelasan Program

Program di atas merupakan implementasi sinkronisasi proses menggunakan semaphore pada sistem operasi Xinu. Terdapat tiga proses, yaitu:

- p1
- p2
- p3

Masing-masing proses bertugas mencetak teks:

- "Kwak"
- "Kwik"
- "Kwek"

secara berurutan dan terus-menerus.

Untuk mengatur urutan eksekusi proses tersebut, digunakan tiga semaphore bertipe `sem_t`, yaitu:

- s1
- s2
- s3

Pada fungsi `main()`, ketiga semaphore diinisialisasi menggunakan `semcreate()`:

```
s1 = semcreate(1);
s2 = semcreate(0);
s3 = semcreate(0);
```

Nilai awal semaphore:

- s1 = 1 → proses p1 dapat berjalan pertama.
- s2 = 0 → proses p2 menunggu giliran.
- s3 = 0 → proses p3 menunggu giliran.

Ketiga proses kemudian dibuat menggunakan `create()` dan dijalankan menggunakan `resume()` sehingga dapat berjalan secara konkuren.

Mekanisme Sinkronisasi

Sinkronisasi dilakukan menggunakan operasi:

- `wait()`
- `signal()`

Penjelasan alur proses:

p1 menjalankan:

```
wait(s1);
```

lalu mencetak:

Kwak

kemudian memberi sinyal:

```
signal(s2);
```

p2 menunggu semaphore s2, lalu mencetak:

Kwik

kemudian memberi sinyal ke s3.

p3 menunggu semaphore s3, lalu mencetak:

Kwek

kemudian memberi sinyal kembali ke s1.

Dengan mekanisme tersebut, proses berjalan secara bergantian dan teratur dengan urutan:

Kwak → Kwik → Kwek

2. Implementasi Producer-Consumer Menggunakan Semaphore

Kode Program

```
#include <xinu.h>
```

```
sid32 empty, full;  
int buffer;
```

```
process producer(void) {
```

```
    int i;
```

```
    for (i = 1; i <= 1000; i++) {
```

```
        wait(empty);
```

```
        buffer = i;
```

```
        printf("Producer memproduksi nilai %d\n", i);
```

```
        signal(full);  
    }
```

```
    return OK;  
}
```

```
process consumer(void) {
```

```
    int i;
```

```
    for (i = 1; i <= 1000; i++) {
```

```
        wait(full);
```

```

        printf("Konsumer menampilkan nilai %d\n", buffer);

        signal(empty);
    }

    return OK;
}

/* ===== MAIN ===== */

process main(void) {

    empty = semcreate(1);
    full = semcreate(0);

    resume(create(producer, 1024, 20, "Producer", 0));
    resume(create(consumer, 1024, 20, "Consumer", 0));

    while (1) {
        sleep(1);
    }

    return OK;
}

```

Penjelasan Program

Program di atas merupakan implementasi masalah klasik *producer-consumer* menggunakan mekanisme sinkronisasi semaphore pada sistem operasi Xinu.

Terdapat dua proses utama, yaitu:

- producer
- consumer

Fungsi Producer

Proses producer bertugas menghasilkan data berupa bilangan dari 1 hingga 1000.

Data tersebut disimpan sementara pada variabel bersama:

```
int buffer;
```

yang digunakan sebagai media komunikasi antar proses.

Fungsi Consumer

Proses consumer bertugas mengambil data dari buffer dan menampilkannya ke layar.

Penggunaan Semaphore

Untuk mengatur akses terhadap buffer agar tidak terjadi konflik data, digunakan dua semaphore bertipe `sid32`, yaitu:

- empty
- full

Pada fungsi main(), kedua semaphore diinisialisasi sebagai berikut:

```
empty = semcreate(1);
full = semcreate(0);
```

Keterangan:

- empty = 1
→ menandakan buffer kosong dan siap diisi.
- full = 0
→ menandakan belum ada data yang dapat dikonsumsi.

Kedua proses kemudian dibuat menggunakan create() dan dijalankan secara konkuren menggunakan resume().

Mekanisme Sinkronisasi

Pada Proses Producer

Sebelum menulis data ke buffer, proses producer menjalankan:

```
wait(empty);
```

untuk memastikan bahwa buffer sedang kosong.

Setelah itu:

1. Data dimasukkan ke buffer.
2. Nilai ditampilkan menggunakan printf().

Producer menjalankan:

```
signal(full);
```

untuk memberi tanda bahwa data siap dikonsumsi.

Pada Proses Consumer

Consumer akan menjalankan:

```
wait(full);
```

untuk menunggu hingga terdapat data di buffer.

Setelah data tersedia:

1. Consumer membaca isi buffer.
2. Menampilkan nilai ke layar.

Menjalankan:

`signal(empty);`

untuk memberi tanda bahwa buffer kembali kosong dan siap diisi oleh producer.

Dengan mekanisme ini, kedua proses dapat berjalan secara bergantian dan terkoordinasi tanpa saling mengganggu.